

Effect of the kappa correction in RBLapSum

Research note: `through_rank_kappa` versus plain `through_rank` at $b_0 = 0$

2026-09-21.

1 Purpose and summary of conclusions

RBLapSum’s boundary surrogate can be applied in two forms. In plain `through_rank`, each candidate’s score gradient is the kernel-weighted boundary term itself. In `through_rank_kappa`, the same term is additionally passed through the LapSum rank-one Jacobian at the boundary,

$$g = a - q \sum_i a_i, \quad q_i = \kappa_i / \sum_j \kappa_j,$$

which removes the common score-translation mode. This note tests whether the kappa correction improves convergence, using matched training runs at a boundary floor of $b_0 = 0$ (the new default; the earlier campaigns used $b_0 = 0.1$).

Conclusions:

- **The kappa correction is decisively better in every tested configuration.** Plain `through_rank` diverges within the first ~ 1700 steps in all four configurations with $K \geq 64$, and in the two $K = 32$ configurations where it survives it converges 0.085–0.164 nats worse than `through_rank_kappa` while showing repeated large excursions along the way.
- **With the correction, five of six configurations converge**, at final validation CE 1.46–1.55; the exception is $(K, J) = (64, 64)$ at $T = 1$, which diverged at step 8680. The $K = 64$, $T = 1$ row is the known marginal row of this surrogate.
- **The boundary floor does not matter in these regimes.** The four `through_rank_kappa` configurations that were previously trained at $b_0 = 0.1$ reproduce at $b_0 = 0$ within ± 0.005 nats, in line with the earlier measurement that the mean boundary stays 8–40 $\times$ above the old floor.

2 Setup

Twelve runs, all under full magnitude–direction decoupling with the same architecture, data, optimizer, schedules, and seed as the concurrent MD-versus-MD-init campaign (8×768 , `abs_topk` over $N = 1536$, `residual_out`, batch 96×512 , 20k steps). The grid pairs the two gradient modes at

$$(K, J, T) \in \{(32, 32, 1), (32, 480, 1), (64, 64, 1), (64, 448, 1), (128, 128, 2), (128, 384, 2)\},$$

with $T = 2$ for the $K = 128$ rows because $K = 128$, $T = 1$ is known to diverge under every regime. All runs use $b_0 = 0$; the run names carry a `b00` marker. Divergence is detected and the run stopped by the same guard as in the companion note (non-finite CE, or CE far above its own best for a sustained window).

3 Results

(K, J)	T	through_rank_kappa	through_rank
(32, 32)	1	1.5533	1.6382
(32, 480)	1	1.4918	1.6558
(64, 64)	1	diverged (8680)	diverged (1580)
(64, 448)	1	1.4776	diverged (1620)
(128, 128)	2	1.4792	diverged (1580)
(128, 384)	2	1.4615	diverged (1680)

Final validation CE after 20k steps, or the step at which the run was stopped as diverged.

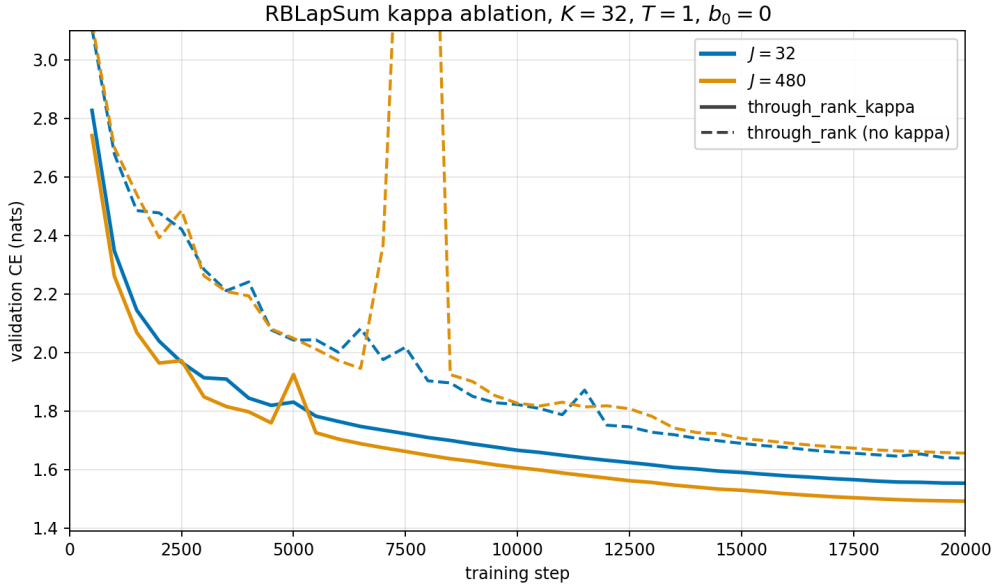


Figure 1: $K = 32$, $T = 1$: both modes survive. The plain `through_rank` runs (dashed) sit ~ 0.1 nats above their kappa counterparts throughout training and undergo repeated large excursions, including a recovered burst of the $J = 480$ run around steps 7000–8500.

Comparison with the earlier $b_0 = 0.1$ runs. Four of the kappa configurations have direct twins in the MD-versus-MD-init campaign, which ran at $b_0 = 0.1$ but is otherwise identical:

(K, J)	T	$b_0 = 0.1$	$b_0 = 0$	difference
(32, 32)	1	1.5482	1.5533	+0.0051
(32, 480)	1	1.4923	1.4918	−0.0005
(128, 128)	2	1.4778	1.4792	+0.0014
(128, 384)	2	1.4602	1.4615	+0.0013

The differences are within run-to-run noise, confirming that the earlier results were not shaped by the floor.

One historical contrast is worth recording: at $(K, J) = (64, 448)$, $T = 1$, the original $b_0 = 0.1$ campaign run destabilized near step 3940, while the present $b_0 = 0$ run converged cleanly. Given the four replication pairs above, this should be read as the known run-to-run stochasticity of divergence in a marginal row rather than as an effect of the floor; the same row also splits here between its $J = 448$ survivor and its $J = 64$ death.

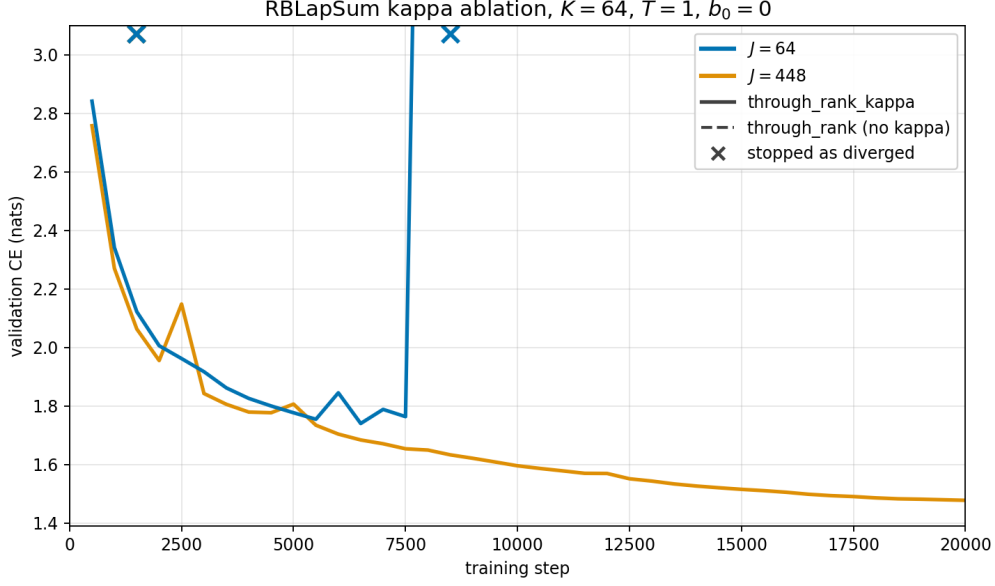


Figure 2: $K = 64, T = 1$: both `through_rank` runs diverge within the first ~ 1600 steps (crosses at the top edge). With the kappa correction, $J = 448$ converges to 1.4776 while $J = 64$ survives to step 8680 before a terminal burst.

4 Conclusion

The kappa correction is not an optional refinement of the `through_rank` surrogate; in these setups it is what makes the surrogate usable. Without it, training survives only in the most conservative row tested ($K = 32, T = 1$), and even there at a 0.085–0.164 nat penalty with visibly bursty dynamics. With it, every configuration except the marginal $(64, 64), T = 1$ cell converges to 1.46–1.55. All cells are single runs at one seed, so individual divergences carry the usual stochastic caveat; the mode-level separation, however, is uniform across all six configurations.

5 Where the mass goes: p_i profiles and total surrogate mass

Hypothesis under test. A natural reading of the no-kappa failure is that the surrogate inflates the soft mass inside the candidate pool: rather than *redistributing* mass across the boundary, the optimizer switches on features that are inactive in the forward pass, so $\sum_i p_i$ grows. This section measures that directly.

The quantity. The LapSum soft assignment for a candidate is

$$p_i = F\left(\frac{s_i - b}{T}\right), \quad F(z) = \begin{cases} \frac{1}{2}e^z, & z \leq 0 \\ 1 - \frac{1}{2}e^{-z}, & z > 0 \end{cases}$$

(the project’s `laplace_cdf`), and $\sum_i p_i$ over the top- $(K + J)$ pool is exactly the quantity LapSum’s barrier solve drives to K . RBLapSum does not solve for the barrier: it pins b to the rank boundary, $b = \max(b_0, s_{(K+1)})$, so here the mass is unconstrained and free to drift. Note that $\sum_i p_i$ is a deterministic function of the score geometry (s, b, T) – it is not an independent degree of freedom, but a readout of that geometry in probability space.

How it was measured. The no-kappa runs died at steps 1620 and 1580, before their first checkpoint at step 2000, so no saved states exist for them. The measurement therefore re-runs

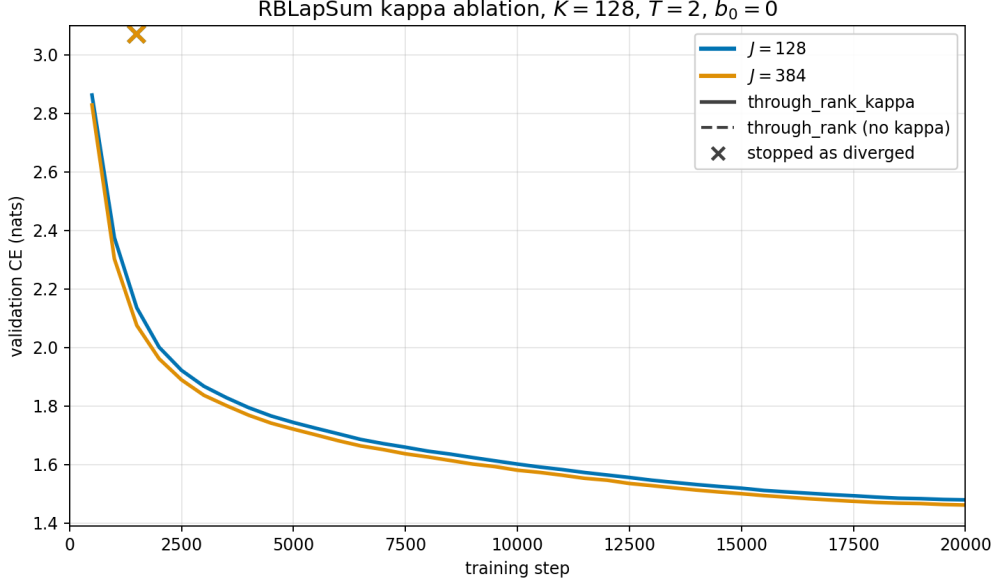


Figure 3: $K = 128, T = 2$: both `through_rank` runs diverge within the first ~ 1700 steps; both kappa runs converge.

each of the four runs from its own dumped `config.json` at the same seed – which reproduces the trajectory – and measures every 50 steps through `train()`’s `on_step` hook, saving no check-points (`analysis/probe_mass.py`). The measurement is a forward pass under `no_grad` on one fixed batch, identical across all four runs, with the gate buffers and RNG state it would perturb snapshotted and restored, so the training trajectory is unaffected. Profiles below are block 4 at one token; the mass was also recorded for all eight blocks at four token positions.

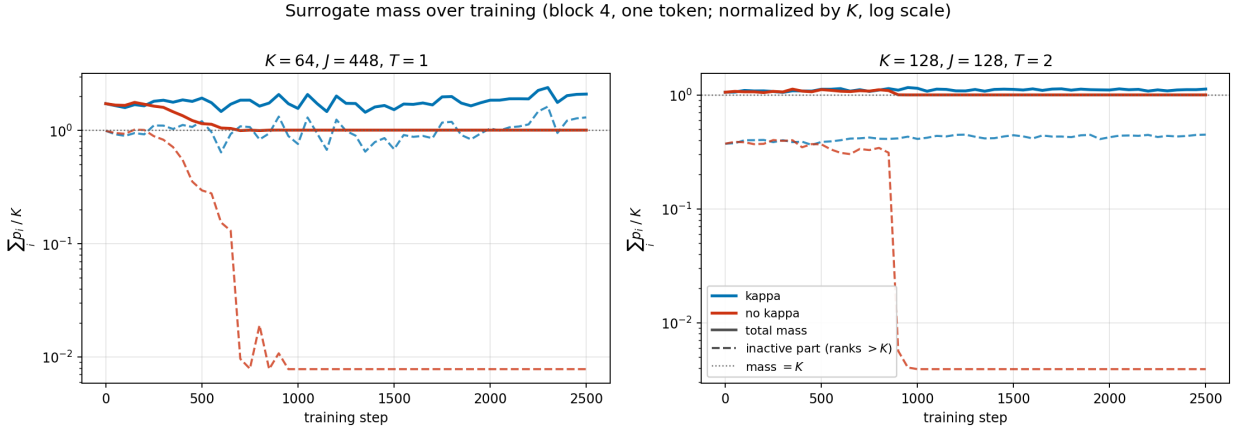


Figure 4: Total surrogate mass $\sum_i p_i$, normalized by K , against training step (log scale); dashed curves are the part carried by the inactive candidates, ranks $> K$. With the kappa correction (blue) the mass stays above K and the inactive part is preserved or grows. Without it (red) the inactive part collapses – to 0.8% of K at $(64, 448)$ and 0.4% at $(128, 128)$ – and the total falls onto exactly $K + \frac{1}{2}$.

Result: the hypothesis is not supported, and the opposite occurs. Without the kappa correction the mass does not grow; it *falls*, and the inactive candidates are switched off rather than on:

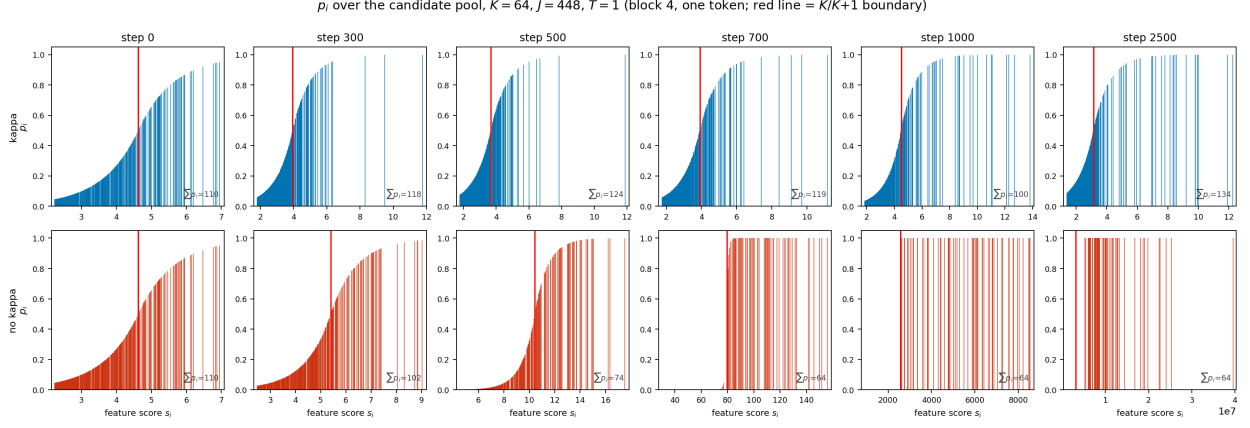


Figure 5: $K = 64$, $J = 448$, $T = 1$: p_i against the feature’s score at six steps, kappa above and plain through_rank below; the red line is the $K/K+1$ boundary. Both start from the same smooth profile. Without the kappa correction it steepens (step 500), becomes a step function by step 700, and thereafter the score axis runs to 10^3 – 10^7 while every active candidate sits at $p_i = 1$ and every inactive one at $p_i \approx 0$. With the correction the profile stays soft for the whole window and the boundary stays inside the transition.

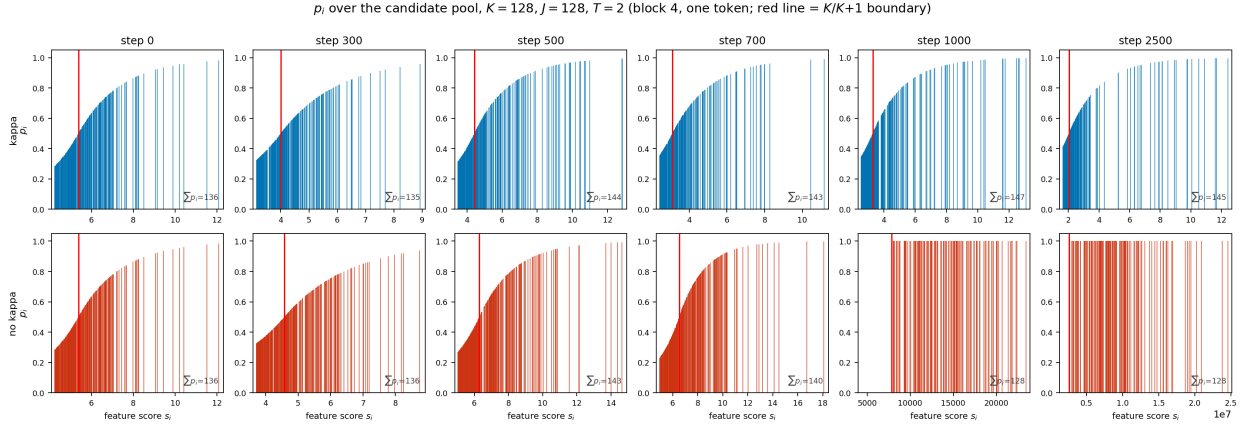


Figure 6: $K = 128$, $J = 128$, $T = 2$: the same comparison. The hardening happens later here (between steps 700 and 1000) and from a narrower starting profile, but ends in the same saturated step.

(K, J)	mode	$\sum p_i / K$ start $\rightarrow$ end	inactive/ K start $\rightarrow$ end	$s_{(1)}$ start $\rightarrow$ end	CE start $\rightarrow$ end
(64, 448)	kappa	1.73 $\rightarrow$ 2.10	0.99 $\rightarrow$ 1.31	6.9 $\rightarrow$ 12	11.3 $\rightarrow$ 1.71
(64, 448)	no kappa	1.73 $\rightarrow$ 1.01	0.99 $\rightarrow$ 0.008	6.9 $\rightarrow 4 \times 10^7$	11.3 $\rightarrow$ 5.85
(128, 128)	kappa	1.06 $\rightarrow$ 1.13	0.37 $\rightarrow$ 0.45	12 $\rightarrow$ 12	11.3 $\rightarrow$ 1.74
(128, 128)	no kappa	1.06 $\rightarrow$ 1.00	0.37 $\rightarrow$ 0.004	12 $\rightarrow 2.5 \times 10^7$	11.3 $\rightarrow$ 7.90

The end state of both no-kappa runs is $\sum_i p_i = K + \frac{1}{2}$ exactly: $p_i = 1$ for the K active candidates, $p_i = \frac{1}{2}$ for the boundary feature itself, and $p_i \approx 0$ for all J inactive ones. This is not specific to the block and token plotted – at the last measured step every one of the eight blocks at all four token positions is within 1% of $(K + \frac{1}{2})/K$ in both no-kappa runs, while no block of either kappa run is.

Timing. The hardening precedes the loss failure rather than following it. At (64, 448) the inactive mass has halved by step 450 and is below 2% of K by step 700, while the training loss only turns upward at step 750. At (128, 128) the two coincide within the 50-step measurement interval (mass collapse at step 900, loss turning at 850).

What the measurement does support. Because $\sum_i p_i$ is fixed by (s, b, T) , the collapse is the score-scale inflation documented in `docs/scale-dynamics-note-neutral.tex`, read out in probability space – and it shows precisely how that inflation disables the surrogate. Once $|s_i - b| \gg T$ for every candidate, F saturates, the kernel $\kappa_i = F'((s_i - b)/T)/T$ vanishes, and the boundary exchange transmits no gradient at all: the run becomes a hard Top- K whose surrogate is numerically dead, which is consistent with its final loss (5.85 and 7.90) being far worse than an actual hard Top- K baseline (1.54 at $K = 64$, 1.49 at $K = 128$). **(M)**

The kappa correction is what prevents this. It subtracts the κ -weighted mean of the candidate gradients, $g = a - q \sum_i a_i$ with $q_i = \kappa_i / \sum_j \kappa_j$, so $L_\kappa \mathbf{1} = 0$ and the common mode – a uniform push on all scores in the pool – is removed by construction. Without it the surrogate retains that component, and a common push is exactly what inflates the score scale relative to the fixed kernel width. This is the interpretation the measurement is consistent with, not something the measurement isolates on its own. **(I)**

6 Gradient structure at the boundary: $\partial L / \partial p_i$ and $\partial L / \partial s_i$

The previous section showed *what* happens to the soft profile without the kappa correction. This section measures the two gradients that drive it, per candidate, on the same runs, block and token.

The quantities. With $y_i = p_i z_i$ the bottleneck output and $u_i = \partial L / \partial y_i$ the gradient arriving at the gate,

$$\frac{\partial L}{\partial p_i} = u_i z_i, \quad a_i = \frac{\partial L}{\partial p_i} \kappa_i, \quad \kappa_i = \frac{\partial p_i}{\partial s_i} = \frac{F'((s_i - b)/T)}{T},$$

and the score gradient is a after the mode correction: `through_rank` removes the common mode as a point mass on the boundary feature, $g = a - (\sum_i a_i) e_{K+1}$, while `through_rank_kappa` spreads it κ -weighted, $g = a - q \sum_i a_i$ with $q_i = \kappa_i / \sum_j \kappa_j$. Both are reported *signed*. Gradient descent moves a quantity against its gradient, so a negative bar means the loss falls if that feature’s gate variable (or score) increases.

The measurement requires a backward pass, so it runs in `train()` mode with `grad` enabled – `surrogate_active()` is `False` in `eval` and under `no_grad`, and an `eval`-mode probe would measure the hard mask instead of the surrogate. The probe’s own gradients are discarded (the hook fires before the step’s `zero_grad`) and the gate buffers and RNG state are restored, as before. Measured every 100 steps to step 1200, which covers both collapses.

Result 1: the point mass makes the boundary feature the loudest in the pool. Taking $|g_{K+1}|$ relative to the largest $|g_i|$ over all other candidates:

step	(64, 448)		(128, 128)	
	kappa	no kappa	kappa	no kappa
0	0.17×	2.0×	0.06×	2.8×
300	1.04×	3.4×	0.04×	6.9×
500	0.02×	0.7×	0.10×	4.5×

Without the correction the boundary feature’s score gradient is two to seven times larger than any other candidate’s; with it, the boundary feature is typically a tenth of the loudest candidate. In absolute terms the difference is one to two orders of magnitude: at step 500, 1.33×10^{-4} against 6.1×10^{-7} at (64, 448), and 1.26×10^{-4} against 2.1×10^{-6} at (128, 128).

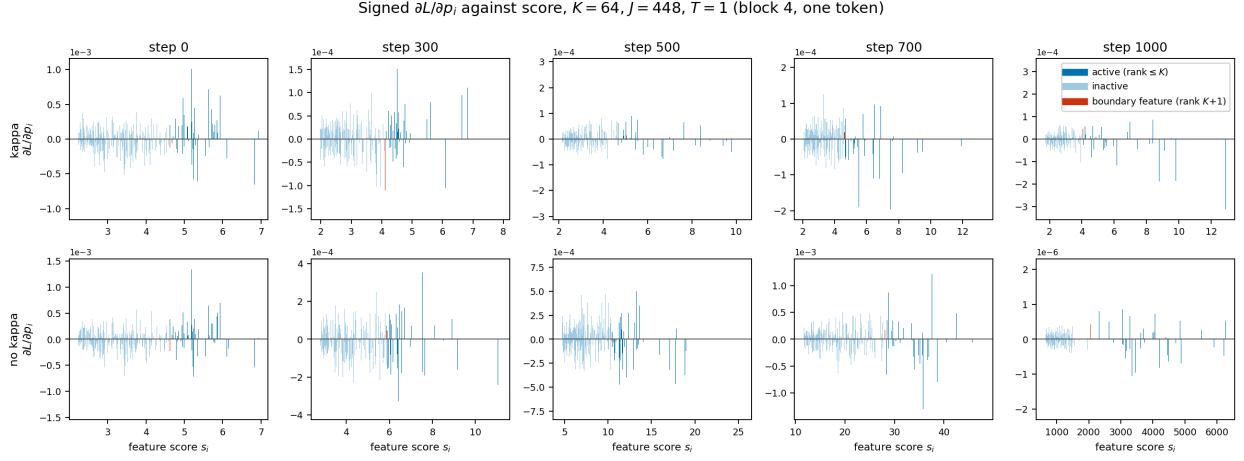


Figure 7: $K = 64, J = 448$: signed $\partial L/\partial p_i$ against the feature’s score. Active candidates are dark blue, inactive light blue, and the boundary feature (rank $K+1$, whose score defines b) red. The two modes look alike: the gradient on the gate variable is broadly distributed over the pool and close to sign-symmetric, and the boundary feature is unremarkable in both. Note the y scale of the last no-kappa panel, 10^{-6} : after the collapse the whole pool’s gradient is three orders of magnitude smaller.

Result 2: after the collapse the score gradient is identically zero. At step 1000 both no-kappa runs have $\partial L/\partial s_i = 0$ for every candidate, to the last bit – visible as an empty panel with a 10^{-30} axis. This is the saturation of the previous section stated in gradient terms: once $|s_i - b| \gg T$ everywhere, κ_i underflows, $a_i = (\partial L/\partial p_i)\kappa_i$ vanishes, and the boundary exchange transmits nothing. The kappa runs keep a score gradient of order 10^{-5} throughout.

Result 3: $\partial L/\partial p_i$ carries no systematic push on the inactive features. The fraction of inactive candidates with $\partial L/\partial p_i < 0$ – the ones the loss would like to switch on – stays at 0.45–0.56 in every run and at every measured step, kappa and no-kappa alike. There is no net demand to activate the inactive part of the pool in either mode, which is a second, independent reason the “mass grows because the optimizer switches inactive neurons on” reading does not hold: the gate-variable gradient is close to sign-symmetric outside the support, and what differs between the modes is not this gradient but how its common mode is routed into the scores.

Reading the two sections together. `through_rank` concentrates the entire common-mode correction on the one feature whose score *is* the boundary. Moving that feature moves b , and hence the reference point of every p_i in the pool; the measured consequence is the score-scale inflation of the previous section, which ends in saturation and a dead surrogate. `through_rank_kappa` distributes the same correction κ -weighted, so no single feature – and in particular not the boundary – receives it as a spike, and the profile stays soft for the whole run. **(M)** for the gradient measurements above; the causal link from the spike to the inflation remains **(I)**, since these are observations along the trajectory rather than an intervention.

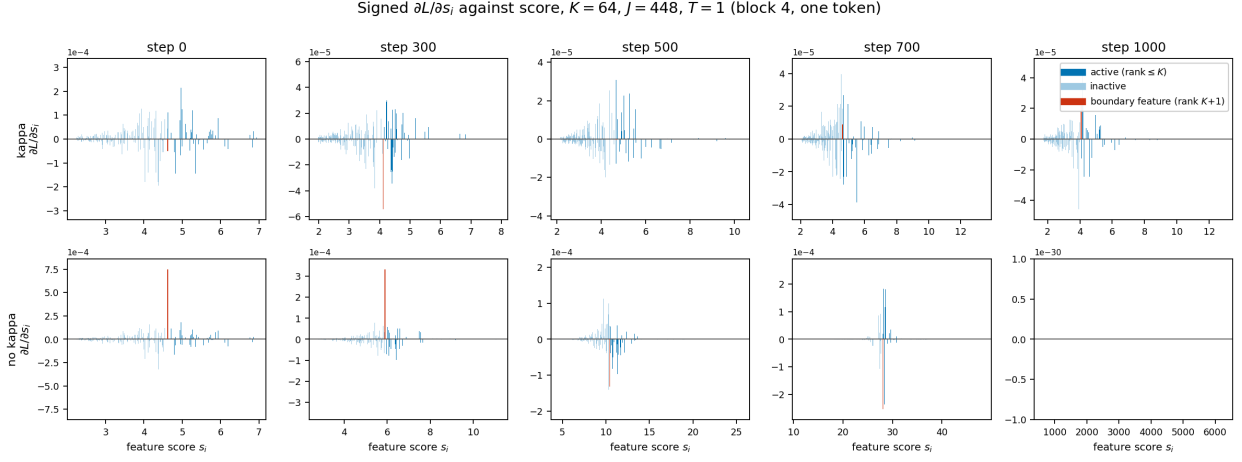


Figure 8: $K = 64, J = 448$: signed $\partial L/\partial s_i$, same encoding. Here the two modes differ sharply. Without the kappa correction the boundary feature carries a single spike that dominates the pool – it is the point mass that removes the common mode – and by step 1000 the panel is empty: the score gradient is exactly zero everywhere. With the correction the same common mode is spread over the pool and the boundary feature is one bar among its neighbours.

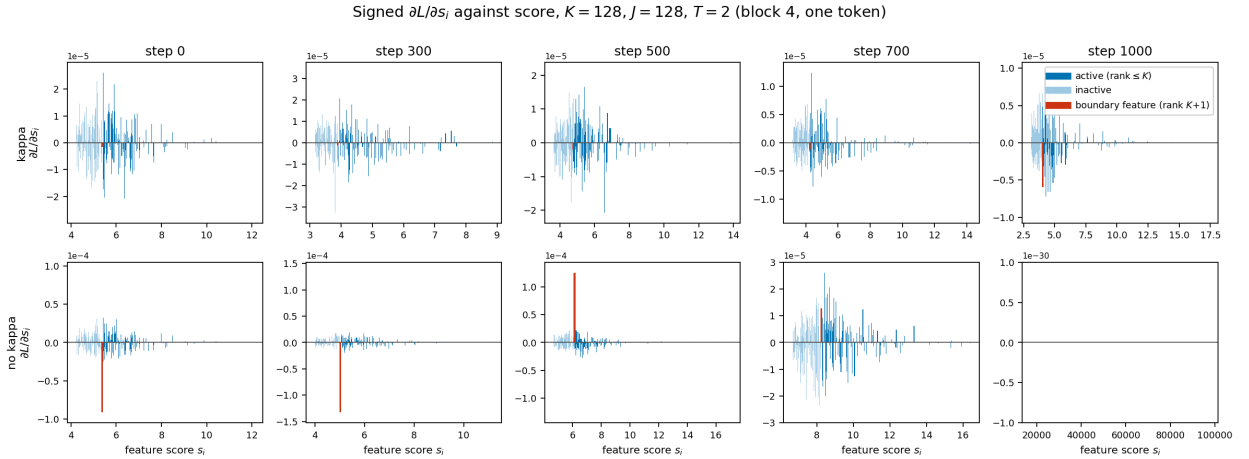


Figure 9: $K = 128, J = 128, T = 2$: the same contrast in $\partial L/\partial s_i$, with the boundary spike up to $6.9\times$ the largest other candidate.